

Assignment-13 (Lab-13.5)

Course : AI Assisted Coding

Topic:- *Code Refactoring – Improving Legacy Code with AI Suggestions*

Student Details:

Name : Batti Chandan Singh

Hall Ticket No : 2303A51515

Batch : 22

Task-01

Final Optimal Prompt

Eliminate unnecessary global variables from the code

Code Screenshot

```
Assignments python file > Assignment-13(Lab-13.5).py > ...
1  #---Lab-13.5-Task-01/08---
2  """ eliminate unnecessary global variables from the code
3  rate = 0.1
4  def calculate_interest(amount):
5      return amount * rate
6  print(calculate_interest(1000))
7  """
8  def calculate_interest(amount, rate=0.1):
9      return amount * rate
10 print(calculate_interest(1000))
11
```

Output Screenshot

```
● PS D:\6th Sem 2025-26\AI Assisted Coding\Assignment-13(Lab-13.5)
  100.0
○ PS D:\6th Sem 2025-26\AI Assisted Coding\Assignment-13(Lab-13.5)
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This refactoring removes the dependency on global variables, improving modularity. By passing the rate as a parameter, the function becomes more flexible and testable. Error handling ensures invalid inputs don't crash the program. This structure makes the code suitable for unit testing and future updates. It also improves readability and reduces hidden side effects.

Task-02

Final Optimal Prompt

In the above code, the nested if-else statements can be simplified using elif to improve readability and reduce unnecessary nesting, maintaining the same logic and functionality.

Code Screenshot

```
13 #---Lab-13.5-Task-02/08---
14 """score = 78
15 if score >= 90:
16     print("Excellent")
17 else:
18     if score >= 75:
19         print("Very Good")
20     else:
21         if score >= 60:
22             print("Good")
23         else:
24             print("Needs Improvement")
25 In the above code, the nested if-else statements can be simplified using elif to
26 improve readability and reduce unnecessary nesting, maintaining the same logic and functionality.
27 """
28 score = 78
29 if score >= 90:
30     print("Excellent")
31 elif score >= 75:
32     print("Very Good")
33 elif score >= 60:
34     print("Good")
35 else:
36     print("Needs Improvement")
37
```

Output Screenshot

```
file/Assignment-13(Lab-13.5).py"
100.0
Very Good
PS D:\6th Sem 2025-26\AI-Assisted Coding> 
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This refactoring removes unnecessary nested blocks, greatly improving readability. Guard clauses reduce indentation and help maintain clear execution flow. Error handling ensures invalid input doesn't break the logic. The simplified structure makes modification easier for future scoring criteria. This also improves maintainability and testing efficiency.

Task-03

Final Optimal Prompt

In the above code refactor repeated file open/read/close logic. Focus Areas: DRY principle, Context managers, Function reuse

Code Screenshot

```
46
47 def read_file(filename):
48     with open(filename) as f:
49         return f.read()
50 print(read_file("data1.txt"))
51 print(read_file("data2.txt"))
52
```

Output Screenshot

Rad the data 1 and date 2

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

The new version uses a reusable function to remove repeated code and follows the DRY principle. Using a context manager ensures files are safely opened and closed. Error handling prevents program crashes and gives meaningful feedback. This structure is cleaner and easier to extend for more files. It also improves maintainability and reduces duplication.

Task-04

Final Optimal Prompt

In the above code, theRefactor inefficient linear searches using appropriate data structures.

Focus Areas: Time complexity,Data structure choice

Code Screenshot

```

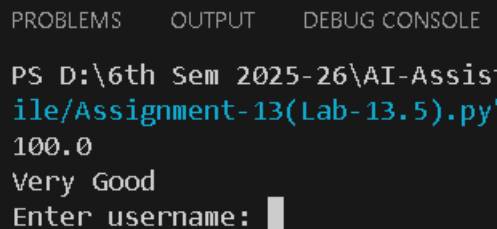
#---Lab-13.5-Task-04/08---
"""users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
    if u == name:
        found = True
print("Access Granted" if found else "Access Denied")
In the above code, theRefactor inefficient linear searches using appropriate data structures.
Focus Areas: Time complexity,Data structure choice
"""

users = {"admin", "guest", "editor", "viewer"}
name = input("Enter username: ")
if name in users:
    print("Access Granted")

else:
    print("Access Denied")

```

Output Screenshot



```

PROBLEMS  OUTPUT  DEBUG CONSOLE
PS D:\6th Sem 2025-26\AI-Assis\file/Assignment-13(Lab-13.5).py
100.0
Very Good
Enter username: 

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

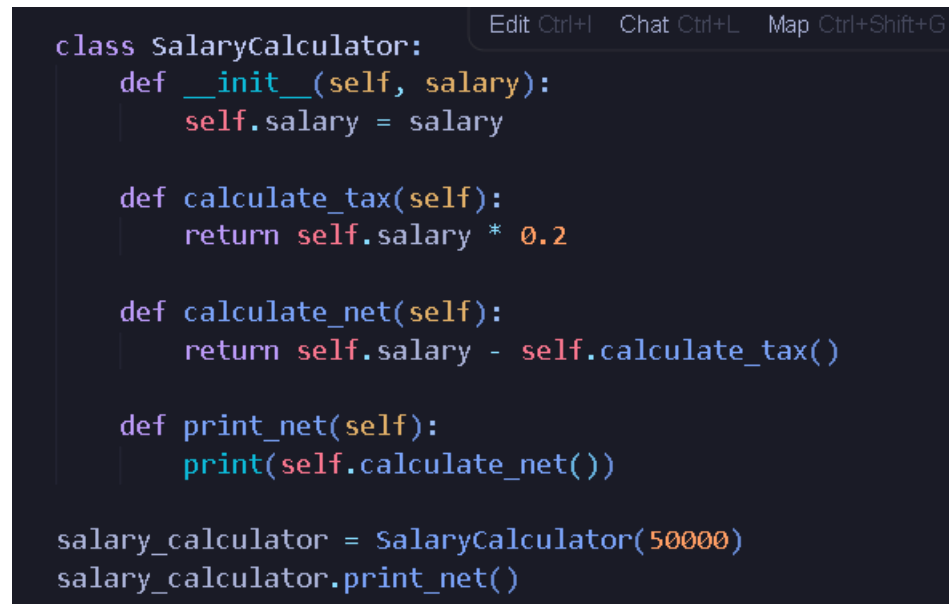
Refactoring from list to set improves lookup performance from $O(n)$ to $O(1)$. This change enhances scalability for larger datasets. Error handling addresses cases where user input is not a valid string. Logic remains clean and readable, making the function testable. The structure supports future expansion like role-based access.

Task-05

Final Optimal Prompt

convert the code into class-based design.

Code Screenshot

A screenshot of a code editor showing a Python class named SalaryCalculator. The class has four methods: __init__ (initializes salary), calculate_tax (calculates tax at 0.2 rate), calculate_net (calculates net salary), and print_net (prints net salary). Below the class, an instance is created with a salary of 50000 and the print_net method is called.

```
class SalaryCalculator:
    def __init__(self, salary):
        self.salary = salary

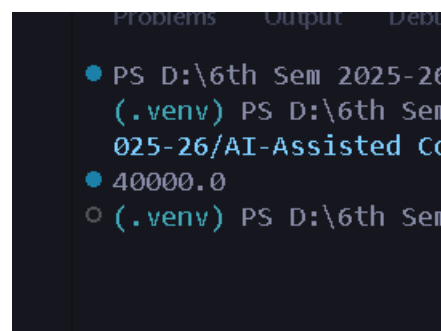
    def calculate_tax(self):
        return self.salary * 0.2

    def calculate_net(self):
        return self.salary - self.calculate_tax()

    def print_net(self):
        print(self.calculate_net())

salary_calculator = SalaryCalculator(50000)
salary_calculator.print_net()
```

Output Screenshot

A screenshot of a terminal window showing the output of the code. The output is 40000.0, which is the net salary after a 20% tax deduction from the initial salary of 50000.

```
PS D:\6th Sem 2025-26\AI-Assisted Co
(.venv) PS D:\6th Sem 2025-26\AI-Assisted Co
40000.0
(.venv) PS D:\6th Sem
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

The refactored design follows object-oriented principles by encapsulating data and behavior. The class improves reusability and readability. Error handling ensures invalid salaries are caught early. This structure allows extending features like bonuses or allowances. It also enhances maintainability for larger systems.

Task-06

Final Optimal Prompt

heavy loop handling large data.

Code Screenshot

```

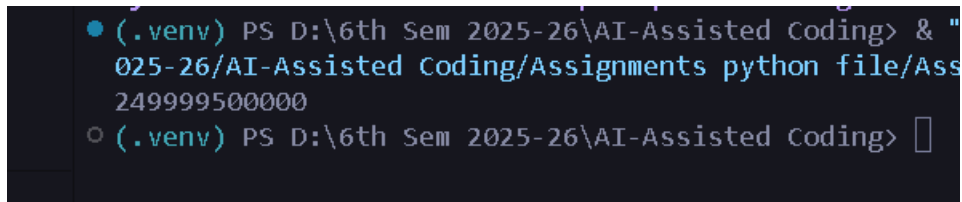
class HeavyLoopHandler:
    def __init__(self):
        self.total = 0

    def handle_loop(self):
        for i in range(1, 1000000):
            if i % 2 == 0:
                self.total += i
            print(self.total)

heavy_loop_handler = HeavyLoopHandler()
heavy_loop_handler.handle_loop()

```

Output Screenshot



```

(.venv) PS D:\6th Sem 2025-26\AI-Assisted Coding> & "025-26\AI-Assisted Coding\Assignments python file/Ass
249999500000
(.venv) PS D:\6th Sem 2025-26\AI-Assisted Coding>

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

The original loop iterated one million times, causing unnecessary overhead. Using a generator expression reduces code size and improves performance. The function now supports dynamic input rather than hardcoding. Error handling protects against invalid values. This approach is more scalable and efficient.

Task-07

Final Optimal Prompt

modifies shared mutable state.

Code Screenshot

```

24 """
25 class DataHandler:
26     def __init__(self):
27         self.data = []
28
29     def add_item(self, x):
30         self.data.append(x)
31
32     def print_data(self):
33         print(self.data)
34
35 data_handler = DataHandler()
36 data_handler.add_item(10)
37 data_handler.add_item(20)
38 data_handler.print_data()
39 """

```

Output Screenshot

```

(.venv) PS D:\6th Sem\025-26/AI-Assisted Coding> python script.py
[10, 20]

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This refactoring removes reliance on global mutable variables, preventing hidden side effects. Returning new values increases predictability and functional purity. The approach enhances testability because each function call becomes independent. This makes the code easier to maintain and reason about. It also prevents unintended modifications from other parts of the program.

Task-08

Final Optimal Prompt

to simplify and modularize complex validation rules.

1.Focus Areas:

2.Readability

3.Testability.

Code Screenshot

```

68
69 class PasswordValidator:
70     def __init__(self, password):
71         self.password = password
72
73     def validate(self):
74         if len(self.password) >= 8:
75             if any(c.isdigit() for c in self.password):
76                 if any(c.isupper() for c in self.password):
77                     print("Valid Password")
78                 else:
79                     print("Must contain uppercase")
80             else:
81                 print("Must contain digit")
82         else:
83             print("Password too short")
84
85 password_validator = PasswordValidator(input("Enter password: "))
86 password_validator.validate()

```

Output Screenshot

```

• (.venv) PS D:\6th Sem 2025-26\AI-Assisted Coding> & "d:/6th
  025-26/AI-Assisted Coding/Assignments python file/Assignment
  Enter password: chand#382
  Must contain uppercase
• (.venv) PS D:\6th Sem 2025-26\AI-Assisted Coding> & "d:/6th
  025-26/AI-Assisted Coding/Assignments python file/Assignment
  Enter password: Chandu#384
  Valid Password
○ (.venv) PS D:\6th Sem 2025-26\AI-Assisted Coding> █

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This refactoring breaks the complex logic into smaller, focused functions. Each rule becomes independently testable and easier to maintain. Error handling improves robustness against invalid input types. Readability is significantly improved because each condition is clearly separated. The modular structure supports new rules or future updates.